NewsReader Ontology

Minutes from the NewsReader workshop held in Amsterdam on May 14th

General

Organization

Antske Fokkens and Selene Kolman

Participants on location (in alphabetical order)

Antske Fokkens, Niels Ockeloen (first half only), Thomas Ploeger, Marco Rospocher, Roxane Segers, Luciano Serafini, Marieke van Erp, Willem van Hage, Chantal van Son, Piek Vossen

Participans via Skype (in alphabetical order)

Itziar Albade, Egoitz Laparra, German Rigau

NewsReader Ontology

Reasoning:

- The end user point of view
- NLP point of view

We should be pragmatic about the ontology: allow for the reasoning we need for use cases and our own analysis.

The end user perspective on the requirements

The end user point of view is the most problematic one:

- How concrete can we make requirements?
    - For information about the user point of view, we should start from the scenarios that we have
    - We need to fill a gap between what we have extracted with the NLP pipeline and what we need for reasoning.
- Which type of information do the scenarios require from the KS?
    - We can look at querying from different angles: Hackathon, Statistics, Synerscope
    - This will provide a collection of experience.

Insights from SynerScope

Our links are triples. They are two way relations: n-ary relations can not be represented (as discussed in the last meeting). Relations can only be established by going through nodes.

What are the implications for ontology?

- Is there a limit to depth of embedding? In theory none, in practice 4
- Multiple inheritance is possible, but you won't get it out while querying the data using the tool (you can select one type at the time)
- Requirement on the data: you have to fill in a type, not a set of types for a layer.

Deciding on what is put on the layers is a task in itself. This can be solved partially by creating joint categories, but that is not always possible (e.g. something being a place and an organization at the same time).

Thomas was pleasantly surprised by the information currently provided: dates, places, geographic locations. At the moment,no specific requests for SynerScope to work with additional information.

Dealing with contradictions

- Quantities and amounts are missing
    - What would we do with this? We use density for dates, but what do we do with amounts?
    - This is more part of crystallizing things after NLP. The example where sources claim a different amount was paid for a dinner can be solved by supposing that someone going to dinner pays something and there is only one payment for one dinner. This way we can find the contradiction and see if we can find out which is the more reliable, or keep both. Reasoning should play a role.
- We have the same for dates. For now we do not have date ranges.

In general, the strategy as outlined for the example above can be applied when we find contradictory data in aggregations. We state some new fact with a new statement, its provenance will state that it is an inferred statement. There are two possibilities for dealing with this:

1. ontological reasoning. From the ontology we can infer things in an easy way (e.g. type information => infer supertypes etc.). This may produce new facts that can be used for visualizing higher-level things (i.e. it can be used for generalizing).

Classification possibilities

We can now only classify on specific events (e.g. *say, buy*). It would be good if we can do this on a higher level (less specific events). This will help to filter our data better.

How do we get to those generic events? It is clear for speech acts. You have an entire forest of parse trees where speech acts are found. You only lift certain groups of trees and only certain verbs with certain other properties that live nearby...

For example: you search for take-overs, involving Volkswagen, then you get qualifiers: *predict/say/claim* then you can plot when the change happens (i.e. when the prediction/expectation becomes a statement or announcement). However, taking place will not be an event itself. The *take-over* is the event.

We want to know how these qualifying verb change over time: expect towards know...
This is not only signaled by speech act or cognitive events: in the beginning there is a lot of uncertainty and later on this reduces or disappears.

We also need to distinguish cognitive events. Communication should be split up in cognition, speech acts etc.

No one has been using the attribution until now. We have to think about what we want to get out of this project, what do we want to know about the world. Whatever we model on events has the be related to what we want to reason about.

Inferencing

So you want to have an ontology that says that 'firing' will change the state of Wiedeking from working for Volkswagen to him not working for Volkswagen. Being employed is one of the properties that can be modeled by events. This requires detailed modeling of the properties of certain concepts.

How do we limit the properties that we need to model
=> by the domain: start with concepts that are particularly relevant for the domain, our specific use cases.

We need to start doing this on a limited scale. It is still an open question as to whether this should be done in the KS.
We can capture this by reasoning ``rules'': for instance, *firing* means:

- precondition: the person is employed
- result: the person is not employed anymore

This means we need rules that can capture a precondition, event and postcondition allowing us to derive new facts and then checking consistency of derived facts.

Often preconditions will be unknown, but checking them is optional. In the end, if someone has been fire, it is contained in the statement that he was employed. Capturing the implications in both ways (firing implies having been employed and no longer being so) would be great, but is the Holy Grail. We should also be content with more messy results or only having such results for a small set of events.

It is probably possible to build machinery and there are WordNet synsets expressing changes, but we don't know which ones and we have adjectives expressing properties. These properties are known and typed. If someone is interested in a property, we need to link the machinery to the property, the machinery can handle changes of the property...

We restrict ourselves to very limited inferencing: *fire* implies previous working there and no longer working there, but no working there means used to make money there, etc. We will not follow paths this long. FrameNet already provides some information about changes that can be used for limited inferences.

There are number of concerns, but probably there is a way to do this in a gradual manner. There will be a lot of additional information if we apply this kind of inferencing to all the data. We can for instance start applying inferencing on a document level. For each document, you can generate a set of facts which are the consequences of the events in the documents. Maybe 40-50 statements, this can be done. Doing this for 20 million triples is not feasible.

An argument for inferencing in the KS: in the KS, we have several variants of a single world. If the KS can slice things in static situations for a given point in time, then we can calculate what is different in the next point of time. If we are doing this, it has to be in the KS.

The visual representation in slices is very difficult at this stage: what granularity should be applied? This depends on the change. There will be cases where users need a blup (not a slice) of some period and there will be cases where users request a specific period in time.
All this information is already available in the KS.

Inference may work both ways: events may imply changes in states, but changes in states can also imply events. It depends on what we can infer (what information do we have). Depending on the user (or use case) we may want to make certain information explicit.

In both cases (change of state => event, event => change of state) one needs to model the events and their implication. The event inferring the property is deductive reasoning: this is the event, so this the consequence. The other way is abductive: you're trying to explain why it happened. It is more complex, because in the first case you have one possible interpretation and the other one leads to many possibilities.

It is important to decide if we are going to do this. It provides possibilities, but is challenging and it has important consequences for the model, the KS and the NLP part.

We should also realize that even though we can infer a lot of things, we can also make use of sources we already have. e.g. FrameNet includes the employment scenario and provides information such as *hiring* must be before *firing* or *quitting*, it even relates the roles of the participants. We can use these resources.

We can assume that there is enough in terms of resources (FrameNet, VerbNet) that we can clean them and structure for this particular domain. We still need to know: do we list all implications per document or for a lot of documents and what do we do with them? Do we want to reason on it in the KS?

Not everything in the KS has a mention. There is no system that is doing this in the SW. There are systems that are able to do this kind of inference in other ways, but these are systems that need to make decisions. The problem is that the data is messy, incomplete and we have to work with this and this is really the challenge. This means that we should start with simple reasoning tasks, e.g. infer the precondition and postcondition from an event and this is a big improvement already. Then we can compare the information we obtain this way to statements in the text. We can restrict more challenging tasks to only NLP.

There are different ways to go step by step. For instance, take data from one year and do this on an instance level and only look at few events.

The main concern at present is how it will be represented in the KS. It is different from what we do now. We do not have such information: no states. This is implicit. If we can solve this we take a few very interesting steps related to use cases.

In our current representation, we have triples attributedTo papers, inferenced facts are attributedTo the reasoner (which can be a system) and there will be a link to the statement it was informed from.

The task involves:

1. recognize the statement
2. write rules that retrieve conditions from the ontology

How to infer can not be expressed in the ontology, but inference rules provide a way to make an ontology like DOLCE. You cannot do this kind of inference using an ontology in the KS, it only provides types and subtypes, but we also have qualities that model changes (see the FrameNet example provided above).

Examples of information currently available in our resources

For fire-10.10 in VerbNet:
http://verbs.colorado.edu/verb-index/vn/fire-10.10.php#fire-10.10
example
"I fired two secretaries from the company."
syntax
Agent V Theme {from} Source
Semantics
cause(Agent, E) location(start(E), Theme, ?Source) not(location(end(E), Theme, ?Source))

Interesting tid bit about VerbNet:
There are semantics for "to floss", stating that the actor takes care of himself/herself, but no semantics to "to buy" or "to invest" etc. How useful.

For "to buy":
http://verbs.colorado.edu/verb-index/vn/get-13.5.1.php#get-13.5.1
semantics
has_possession(start(E), ?Source, Theme)
transfer(during(E), Theme)
has_possession(end(E), Agent, Theme)
cause(Agent, E)
For "to invest"
http://verbs.colorado.edu/verb-index/vn/equip-13.4.2.php#equip-13.4.2
semantics
has_possession(start(E), Agent, Theme)
has_possession(end(E), Recipient, Theme)
transfer(during(E), Theme)
cause(Agent, E)
I stand/sit corrected.

Overview of information we want to work with

- definitions of roles: follows from the property definitions
- definitions of event duration: important for time line reconstruction
- definition of causation will have impact on the roles (but we leave it for year 3)
- definition of *necessary* (obligatory) participants
- event types:
    - communication & cognition (will be more detailed, but more general than the predicates themselves)
        - in relation to our attribution models
        - cognition
            - planning something, thinking about something
        - speech acts

- other
  - changes of properties (quality regions)
  - specification of properties
    - property = beingEmployed
    - property = beingOwned
    - property = haveCash
  - pre & post conditions of events:
    - worksFor = property
    - CHANGE-JOB
      - HAS-PRECONDITION X worksFor Y
      - HAS-POSTCONDITION not(X worksFor Y)
      - HAS-PARTICIPANT X
      - HAS-PARTICIPANT Y
    - *fire* subClassOf CHANGE-JOB
    - *leave* subClassOf CHANGE-JOB
  - selectional preferences on the participants
    - if only used in NLP then we don't need it in the KS
    - if we find out that it is necessary to reason over the resulting dataset, then it needs to be in there.
  - we only need subtypes as far as they reflect different pre&post condition in relation to the properties we are interested in: in the case of employedBy we can have a single event type for CHANGE_JOB with the pre/post condition (with FIRE and QUIT to the subtypes of CHANGE_JOB) since the difference is not related to the property reasoning. If we are in addition interested into BAD and GOOD changes, fire can be mapped again to FEEL_BAD in addition to the previous mapping.

According to SUMO: mcr:TerminatingEmployment or mcr:LeavingAnOrganization

NLP requirements on the ontology

Goals

- excluding errors
- resolving ambiguity
- resolving metonymy: people/organization, company/product or country-place/country-government
- acquiring missing types

Requirements for the ontology

- selectional preferences and types of participants that match these preferences
- relations between concepts that explain metonymic usage:
    - constituency of types of entities: country has government, football team, etc.
- relations between concepts that explain type coercion -> "enjoy the coffee" ... implicit events ("enjoy drinking the coffee") ... are we only "enjoying" events?
- ontology shared by all languages
- ontology linked to WordNets

Note: metonymy where a non-event denoting noun is implying an event, it will be annotated as an event according to the annotation guidelines. It is not clear, if it will apply to the enjoying coffee example by Pustejovsky, but a bomb will be annotated as the mention of an explosion or attack if the context entails this interpretation.

Available data and resources

- FrameNet
  https://framenet.icsi.berkeley.edu/fndrupal/about
  http://rhizomik.net/html/framenetonto/
  http://www.ontologydesignpatterns.org/ont/framenet/
  https://framenet.icsi.berkeley.edu/fndrupal/FrameGrapher
- Predicate matrix
  http://adimen.si.ehu.es/web/PredicateMatrix NEW Version 1.1! Much larger and rich! now includes many relations across different frames that can be used to make inferences:
    - implicit knowledge
    - implicit roles
    - implicit events
- Wordnets
- DOLCE, SUMO
- DBpedia, schema.org
- VerbNet

Particularly interesting types: *takes_over*, *acquisitions*, *management_changes*, *market_shares*, *production_volumes*, *ownership _of_companies* ...

Action plan

1. look at the most frequent events in data
2. define the set of relevant properties
3. manually define event TYPES and properties in OWL schema
4. select the most abstract generic frames from FrameNet/Predicate Matrix (maybe other categories are relevant)

5. use the Predicate Matrix to map vocabulary to the TYPES (also look at WordNet lexicographers files, SUMO, etc. ... to get maximum coverage)

Event team: Roxane, Egoitz, German, Luciano, Marco, Marieke, Antske
Entity team: Roxane, Egoitz, German, Luciano, Marco, Marieke, Antske

Modeling Issues

Entities, Roles versus Types (see ISWC paper)

nwr:kill sem:hasActor dbp:Wiedeking ;
fn:Kill#Actor dbp:Wiedeking .

nwr:kill nwr:fnKiller dbp:Wiedeking
nwr:fnKiller rdfs:subPropertyOf sem:hasActor

Instances and types (*products*, *software*)

The solution can possibly be found in metonymy:

- *Maverick* is an instance of software product or an artefact (like Guarino)
- *Maverick copy* is instance of software copy

The first case (the software product) has some properties that point toward it being a concept (e.g. the copies being the mentions) as well, but the approach proposed above can address this issue.

Another problem

- A Ford gets a unique instance URI and not the dbp:Ford URI
  <nwr:jfhgjfgsfgfgjfdjg#Ford>
  a sem:Actor
  nwr:copyOf dbp:Ford
- 100.000 Ford cars produced gets a unique instance URI and not dbp:Ford and not 100.000 instances
  <nwr:jfhgfdsghfgklfgf#100,000Fords>
  a sem:Actor, nwr:Group

This brings us back to one of our original user requirements that are currently missing: we need to be able to model quantities.